MakeMyMarrige

Complete Product Repository & Folder Structure

Engineering blueprint • TypeScript monorepo • Modular monolith

Technology baseline

Next.js + TypeScript PWA · NestJS REST API · MongoDB Atlas · Redis + BullMQ · Private Cloudflare R2 · OpenAPI 3.1 · Separate web, API and worker processes

This document consolidates the repository tree, frontend and backend structure, worker, shared packages, infrastructure, testing, conventions and implementation sequence.

1. Architecture overview

Use one repository with three deployable applications. The web app serves the user interface, the API handles synchronous business operations, and the worker executes asynchronous jobs. Domain modules remain inside one modular-monolith API; do not split them into microservices for the pilot.

Core rule: separate applications at runtime, share a repository and carefully bounded packages.

2. Root repository — makemymarrige/

```
makemymarrige/
■
■■■ apps/
■   ■■■ web/                        # Next.js frontend
■   ■■■ api/                        # NestJS REST API
■   ■■■ worker/                     # BullMQ background workers
■
■■■ packages/
■   ■■■ contracts/                  # Shared API contracts and DTO types
■   ■■■ config/                     # Shared TypeScript, ESLint, etc.
■   ■■■ logger/                     # Shared logging utilities
■   ■■■ observability/              # Tracing and metrics helpers
■   ■■■ shared/                     # Small, domain-neutral utilities
■
■■■ docs/
■   ■■■ product/PRD.md
■   ■■■ architecture/SYSTEM_DESIGN.md
■   ■■■ database/
■   ■   ■■■ DATABASE_DESIGN.md
■   ■   ■■■ ER_DIAGRAMS/
■   ■■■ api/
■   ■   ■■■ openapi.yaml
■   ■   ■■■ openapi.json
■   ■   ■■■ API_DOCUMENTATION.md
■   ■■■ workflows/
■   ■■■ security/
■   ■■■ decisions/                  # Architecture Decision Records
■   ■■■ runbooks/
■
■■■ infrastructure/
■   ■■■ docker/
■   ■■■ cloudflare/
■   ■■■ deployment/
■   ■■■ monitoring/
■
■■■ scripts/
■   ■■■ setup/
■   ■■■ database/
■   ■■■ development/
■   ■■■ release/
■
■■■ tests/
■   ■■■ e2e/
■   ■■■ contract/
■   ■■■ security/
■   ■■■ fixtures/
■
■■■ .github/workflows/
■■■ .env.example
■■■ .gitignore
■■■ .editorconfig
■■■ package.json
■■■ pnpm-workspace.yaml
■■■ pnpm-lock.yaml
■■■ turbo.json
■■■ tsconfig.base.json
■■■ eslint.config.mjs
■■■ prettier.config.mjs
■■■ commitlint.config.js
■■■ README.md
```

3. Frontend — apps/web/

Use Next.js App Router. Organise by features. Each feature owns its UI, hooks, validation and frontend service; reusable primitives belong in components/ui.

```
apps/web/
■■■ public/
■   ■■■ images/
■   ■■■ icons/
■   ■■■ fonts/
■   ■■■ manifest.json
■   ■■■ favicon.ico
■
■■■ src/
■   ■■■ app/
■   ■   ■■■ (marketing)/
■   ■   ■   ■■■ page.tsx
■   ■   ■   ■■■ about/
```

```
■   ■   ■   ■■■ contact/
■   ■   ■■■ (auth)/
■   ■   ■   ■■■ login/
■   ■   ■   ■■■ register/
■   ■   ■   ■■■ verify-email/
■   ■   ■   ■■■ forgot-password/
■   ■   ■   ■■■ reset-password/
■   ■   ■■■ (workspace)/
■   ■   ■   ■■■ wedding/[weddingId]/
■   ■   ■       ■■■ layout.tsx
■   ■   ■       ■■■ dashboard/
■   ■   ■       ■■■ functions/
■   ■   ■       ■■■ tasks/
■   ■   ■       ■■■ guests/
■   ■   ■       ■■■ households/
■   ■   ■       ■■■ rsvp/
■   ■   ■       ■■■ budget/
■   ■   ■       ■■■ expenses/
■   ■   ■       ■■■ vendors/
■   ■   ■       ■■■ shagun/
■   ■   ■       ■■■ gallery/
■   ■   ■       ■■■ notifications/
■   ■   ■       ■■■ audit/
■   ■   ■       ■■■ organisers/
■   ■   ■       ■■■ settings/
■   ■   ■■■ (guest)/
■   ■   ■   ■■■ w/[publicWeddingSlug]/
■   ■   ■       ■■■ page.tsx
■   ■   ■       ■■■ functions/
■   ■   ■       ■■■ rsvp/
■   ■   ■       ■■■ gallery/
■   ■   ■       ■■■ shagun/
■   ■   ■■■ invitations/
■   ■   ■■■ forbidden/
■   ■   ■■■ not-found.tsx
■   ■   ■■■ error.tsx
■   ■   ■■■ global-error.tsx
■   ■   ■■■ layout.tsx
■   ■   ■■■ globals.css
■   ■
■   ■■■ features/
■   ■   ■■■ auth/          ■■■ wedding/
■   ■   ■■■ dashboard/     ■■■ governance/
■   ■   ■■■ functions/     ■■■ tasks/
■   ■   ■■■ guests/        ■■■ households/
■   ■   ■■■ rsvp/          ■■■ budget/
■   ■   ■■■ expenses/      ■■■ vendors/
■   ■   ■■■ shagun/        ■■■ gallery/
■   ■   ■■■ notifications/ ■■■ audit/
■   ■   ■■■ workspace-settings/
■   ■
■   ■■■ components/
■   ■   ■■■ ui/            # Reusable design-system primitives
■   ■   ■■■ layout/
■   ■   ■■■ navigation/
■   ■   ■■■ forms/
■   ■   ■■■ data-display/
■   ■   ■■■ feedback/
■   ■■■ hooks/
■   ■■■ lib/
■   ■   ■■■ api/
■   ■   ■■■ auth/
■   ■   ■■■ permissions/
■   ■   ■■■ validation/
■   ■   ■■■ formatting/
■   ■   ■■■ utilities/
■   ■■■ providers/
■   ■■■ stores/
■   ■■■ types/
■   ■■■ middleware.ts
■
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■■■ next.config.ts
■■■ package.json
■■■ tsconfig.json
```

4. Frontend feature example — guests/

```
features/guests/
■■■ components/
■   ■■■ GuestList.tsx
■   ■■■ GuestForm.tsx
■   ■■■ GuestDetails.tsx
■   ■■■ GuestFilters.tsx
■■■ hooks/
■   ■■■ useGuests.ts
■   ■■■ useGuestDetails.ts
■■■ services/guestService.ts
■■■ schemas/guestSchema.ts
■■■ types/guest.types.ts
■■■ index.ts
```

5. Backend — apps/api/

Use NestJS modules organised around business domains. Each module owns business rules, application use cases and persistence logic.

```
apps/api/
■■■ src/
■   ■■■ main.ts
■   ■■■ app.module.ts
■   ■■■ config/
■   ■   ■■■ configuration.ts
■   ■   ■■■ validation.schema.ts
■   ■   ■■■ environment.ts
■   ■■■ common/
■   ■   ■■■ decorators/
■   ■   ■■■ guards/
■   ■   ■■■ interceptors/
■   ■   ■■■ filters/
■   ■   ■■■ pipes/
■   ■   ■■■ middleware/
■   ■   ■■■ exceptions/
■   ■   ■■■ constants/
■   ■   ■■■ utils/
■   ■■■ infrastructure/
■   ■   ■■■ database/
■   ■   ■   ■■■ mongodb/
■   ■   ■   ■■■ transactions/
■   ■   ■   ■■■ indexes/
■   ■   ■■■ redis/
■   ■   ■■■ storage/r2/
■   ■   ■■■ email/
■   ■   ■■■ payments/
■   ■   ■■■ maps/
■   ■   ■■■ observability/
■   ■■■ modules/
■   ■   ■■■ auth/
■   ■   ■■■ users/
■   ■   ■■■ weddings/
■   ■   ■■■ memberships/
■   ■   ■■■ invitations/
■   ■   ■■■ governance/
■   ■   ■■■ functions/
■   ■   ■■■ tasks/
■   ■   ■■■ guests/
■   ■   ■■■ households/
■   ■   ■■■ rsvp/
■   ■   ■■■ budget/
■   ■   ■■■ expenses/
■   ■   ■■■ vendors/
■   ■   ■■■ shagun/
■   ■   ■■■ photos/
■   ■   ■■■ notifications/
■   ■   ■■■ dashboard/
■   ■   ■■■ audit/
■   ■   ■■■ sessions/
■   ■   ■■■ security-incidents/
■   ■   ■■■ workspace-lifecycle/
■   ■■■ events/
■   ■   ■■■ outbox/
■   ■   ■■■ inbox/
■   ■   ■■■ domain-events/
■   ■■■ health/
■■■ test/
■   ■■■ unit/
■   ■■■ integration/
■   ■■■ e2e/
■■■ nest-cli.json
■■■ package.json
■■■ tsconfig.json
```

6. Backend module template — modules/guests/

```
modules/guests/
■■■ guests.module.ts
■■■ presentation/
■   ■■■ guests.controller.ts
■   ■■■ guests.admin.controller.ts
■   ■■■ dto/
■       ■■■ create-guest.dto.ts
■       ■■■ update-guest.dto.ts
■       ■■■ guest-query.dto.ts
■■■ application/
■   ■■■ use-cases/
■   ■   ■■■ create-guest.use-case.ts
■   ■   ■■■ update-guest.use-case.ts
■   ■   ■■■ archive-guest.use-case.ts
■   ■   ■■■ get-guest.use-case.ts
■   ■■■ services/
■   ■■■ ports/
■■■ domain/
■   ■■■ entities/
■   ■■■ value-objects/
■   ■■■ events/
■   ■■■ policies/
■   ■■■ enums/
■   ■■■ errors/
■■■ infrastructure/
■   ■■■ persistence/
■   ■   ■■■ guest.schema.ts
■   ■   ■■■ guest.repository.ts
■   ■   ■■■ guest.mapper.ts
■   ■■■ adapters/
■■■ guests.module.spec.ts
```

Backend layer responsibilities

Layer	Responsibility
Presentation	HTTP controllers, request DTOs and response mapping
Application	Use cases and orchestration
Domain	Business rules, entities, policies and state transitions
Infrastructure	MongoDB, Redis and external provider implementations

Do not put business logic inside controllers. A controller receives and validates the request, invokes a use case and returns the result.

7. Background worker — apps/worker/

The API writes durable events to MongoDB's transactional outbox. A dispatcher publishes eligible jobs to BullMQ. Workers process them idempotently. BullMQ is the execution queue, not your source of truth.

```
apps/worker/
■■■ src/
■   ■■■ main.ts
■   ■■■ worker.module.ts
■   ■■■ infrastructure/
■   ■   ■■■ bullmq/
■   ■   ■■■ redis/
■   ■   ■■■ observability/
■   ■■■ processors/
■   ■   ■■■ email/
■   ■   ■■■ notifications/
■   ■   ■■■ photos/
■   ■   ■■■ shagun/
■   ■   ■■■ workspace-deletion/
■   ■   ■■■ invitations/
■   ■   ■■■ maintenance/
■   ■■■ schedulers/
■   ■   ■■■ wedding-lifecycle.scheduler.ts
■   ■   ■■■ maintenance.scheduler.ts
■   ■■■ health/
■■■ test/
■■■ package.json
```

8. Shared packages — packages/

Keep shared packages small. Domain-specific logic should stay with its owning module.

```
packages/
■■■ contracts/
■   ■■■ src/
```

```
■   ■   ■■■ api/
■   ■   ■■■ schemas/
■   ■   ■■■ enums/
■   ■   ■■■ index.ts
■   ■■■ package.json
■■■ config/
■   ■■■ eslint/
■   ■■■ typescript/
■   ■■■ package.json
■■■ logger/
■   ■■■ src/
■   ■■■ package.json
■■■ observability/
■   ■■■ src/
■   ■■■ package.json
■■■ shared/
    ■■■ src/
    ■   ■■■ types/
    ■   ■■■ constants/
    ■   ■■■ utilities/
    ■■■ package.json
```

9. Infrastructure/

```
infrastructure/
■■■ docker/
■   ■■■ Dockerfile.web
■   ■■■ Dockerfile.api
■   ■■■ Dockerfile.worker
■   ■■■ docker-compose.yml
■■■ cloudflare/
■   ■■■ r2/
■   ■■■ security/
■■■ deployment/
■   ■■■ staging/
■   ■■■ production/
■■■ monitoring/
    ■■■ dashboards/
    ■■■ alerts/
```

10. Test structure/

Cross-tenant access tests are particularly important. Every tenant-owned query and mutation must prove that a user cannot access another wedding's data by manipulating an identifier.

```
tests/
■■■ contract/
■   ■■■ openapi/
■   ■■■ api-contract/
■■■ e2e/
■   ■■■ authentication/
■   ■■■ tenant-isolation/
■   ■■■ governance/
■   ■■■ finance/
■   ■■■ rsvp/
■   ■■■ lifecycle/
■■■ security/
■   ■■■ authorization/
■   ■■■ cross-tenant-access/
■   ■■■ session-security/
■■■ fixtures/
```

11. Development conventions

Concern	Recommendation
Monorepo tooling	pnpm workspaces + Turborepo
Frontend	Next.js App Router
Backend	NestJS modular monolith
Worker	Separate NestJS application
Validation	Zod on frontend; class-validator or Zod-based DTO validation on API
API contract	OpenAPI 3.1
Database access	Mongoose or MongoDB driver, consistently applied
Testing	Jest/Vitest, Supertest and Playwright
Formatting	ESLint + Prettier
CI/CD	GitHub Actions
Containerisation	Docker
Logging	Structured JSON logs
Version control	Feature branches + protected main

12. Recommended implementation sequence

Phase 1 — Repository foundation

Monorepo, pnpm, TypeScript configuration, linting, formatting, environment validation and CI.

Phase 2 — Backend foundation

NestJS bootstrap, MongoDB connection, Redis connection, exception handling, logging, health checks and API documentation.

Phase 3 — Identity and tenancy

Accounts, authentication, sessions, memberships, invitations and tenant isolation.

Phase 4 — Core wedding operations

Governance, functions, tasks, households, guests and RSVP.

Phase 5 — Finance and vendors

Budget, expenses, Treasurer workflows, vendor discovery and Shagun provider integration.

Phase 6 — Media and engagement

Photo uploads, moderation, notifications and dashboards.

Phase 7 — Security and release readiness

Audit, workspace deletion, penetration testing, backups, monitoring and deployment.

13. Key architectural guardrails

• **Three deployable applications, one repository:** web, API and worker.

• **Modular monolith:** wedding, guest, finance and RSVP remain modules inside the same API application.

• **Tenant isolation:** derive wedding context from authenticated membership; never trust a client-supplied weddingId as authority.

• **Durable asynchronous work:** commit business events to the MongoDB outbox before dispatching to BullMQ.

• **Idempotency:** workers and webhook handlers must safely tolerate retries and duplicate delivery.

- **Source of truth:** MongoDB stores durable business state; Redis is cache/queue infrastructure, not the source of truth.
- **Security:** authorise every tenant-owned read and write, including signed photo URL issuance and background jobs.

14. Scope note

This is a recommended repository blueprint, not a generated application. The folders are architectural placements; they do not mean the files, endpoints, integrations, tests or infrastructure have already been implemented. Create folders as the corresponding phase begins, and keep the code aligned with the PRD, system design, database design and OpenAPI contract.